

Sri Sivasubramaniya Nadar College of Engineering, Chennai
(An autonomous Institution affiliated to Anna University)

Degree & Branch	B.E. Computer Science & Engineering	Semester	V
Subject Code & Name	ICS1512 & Machine Learning Algorithms Laboratory		
Academic year	2025-2026 (Odd)	Batch:2023-2028	Due date:

Experiment 4: Ensemble Prediction and Decision Tree Model Evaluation

1 Aim:

To implement and evaluate multiple machine learning classifiers—including Decision Tree, AdaBoost, Gradient Boosting, XGBoost, Random Forest, and Stacked Ensemble—on the Wisconsin Diagnostic Dataset, optimize their performance through hyperparameter tuning, and compare their predictive accuracy and generalization using 5-Fold Cross-Validation.

2 Libraries used:

- Numpy
- Pandas
- Matplotlib
- Scikit-learn
- Seaborn

3 Objective:

To build classifiers such as Decision Tree, AdaBoost, Gradient Boosting, XGBoost, Random Forest, and Stacked Models (using SVM, Naive Bayes, Decision Tree) and evaluate their performance through 5-Fold Cross-Validation and hyperparameter tuning.

4 Codes:

Decision_Tree

August 29, 2025

```
[3]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV,
    ↳StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score, roc_auc_score,
    ↳roc_curve
)
import matplotlib.pyplot as plt
```

```
[4]: # 1. Load dataset
# wdbc.data does not have headers, so we define them
columns = ["ID", "Diagnosis"] + [f"feature_{i}" for i in range(1, 31)]
data = pd.read_csv("wdbc.data", header=None, names=columns)
```

```
[5]: # 2. Prepare features and target
X = data.drop(["ID", "Diagnosis"], axis=1)
y = data["Diagnosis"].map({"M": 1, "B": 0}) # Malignant=1, Benign=0
```

```
[6]: # 4. Preprocessor (scaling not needed for trees, but kept for pipeline
    ↳consistency)
num_features = X.columns.tolist()
preprocessor = ColumnTransformer(
    transformers=[("scale", StandardScaler(), num_features)],
    remainder="drop"
)
```

```
[7]: X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
[8]: # 5. Pipeline
pipe = Pipeline([
    ("prep", preprocessor),
```

```
    ("clf", DecisionTreeClassifier(random_state=42))
])
```

[9]: *# 6. Hyperparameter grid*

```
param_grid = {
    "clf__criterion": ["gini", "entropy", "log_loss"],
    "clf__max_depth": [3, 5, 10],
    "clf__min_samples_split": [2, 5, 10],
    "clf__min_samples_leaf": [1, 2, 4],
}
```

```
[10]: cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
grid = GridSearchCV(pipe, param_grid, cv=cv, scoring="roc_auc", n_jobs=-1,
    ↪refit=True)
grid.fit(X_train, y_train)
```

```
print("Best Hyperparameters:", grid.best_params_)
print("Best Mean CV AUC:", grid.best_score_)
```

Best Hyperparameters: {'clf__criterion': 'gini', 'clf__max_depth': 5,
'clf__min_samples_leaf': 4, 'clf__min_samples_split': 10}
Best Mean CV AUC: 0.9558823529411764

[11]: *# 8. Evaluate on test set*

```
best_model = grid.best_estimator_
y_pred = best_model.predict(X_test)
y_proba = best_model.predict_proba(X_test)[: , 1]
```

```
[12]: print("\nTest Accuracy:", accuracy_score(y_test, y_pred))
print("Test Precision:", precision_score(y_test, y_pred))
print("Test Recall:", recall_score(y_test, y_pred))
print("Test F1:", f1_score(y_test, y_pred))
print("Test ROC AUC:", roc_auc_score(y_test, y_proba))
```

Test Accuracy: 0.8771929824561403
Test Precision: 0.9117647058823529
Test Recall: 0.7380952380952381
Test F1: 0.8157894736842105
Test ROC AUC: 0.9654431216931217

```
[13]: import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix

# ---- Class distribution ----
plt.figure(figsize=(5,4))
sns.countplot(x=y, palette="coolwarm")
```

```

plt.xticks([0,1], ["Benign (0)", "Malignant (1)"])
plt.title("Class Distribution (WDBC)")
plt.xlabel("Diagnosis")
plt.ylabel("Count")
plt.show()

# ---- Feature importance (from Decision Tree) ----
best_clf = best_model.named_steps["clf"]
importances = best_clf.feature_importances_
features = X.columns

# Sort feature importances
indices = np.argsort(importances)[::-1]
plt.figure(figsize=(10,6))
sns.barplot(x=importances[indices], y=features[indices], palette="viridis")
plt.title("Feature Importance - Decision Tree")
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.show()

# ---- Confusion Matrix ----
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False,
            xticklabels=["Benign (0)", "Malignant (1)"],
            yticklabels=["Benign (0)", "Malignant (1)"])
plt.title("Confusion Matrix - Decision Tree")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()

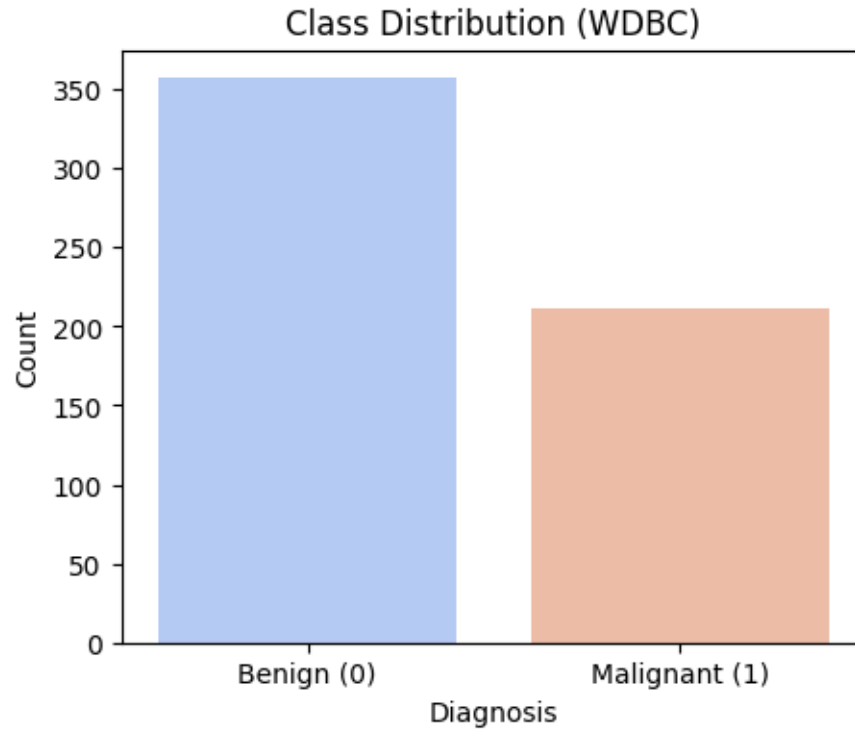
# ---- ROC Curve (already included) ----
fpr, tpr, _ = roc_curve(y_test, y_proba)
plt.figure(figsize=(6,4))
plt.plot(fpr, tpr, label="Decision Tree", linewidth=2)
plt.plot([0, 1], [0, 1], linestyle="--", color="gray")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Decision Tree (WDBC)")
plt.legend()
plt.show()

```

/tmp/ipykernel_7915/1817741554.py:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

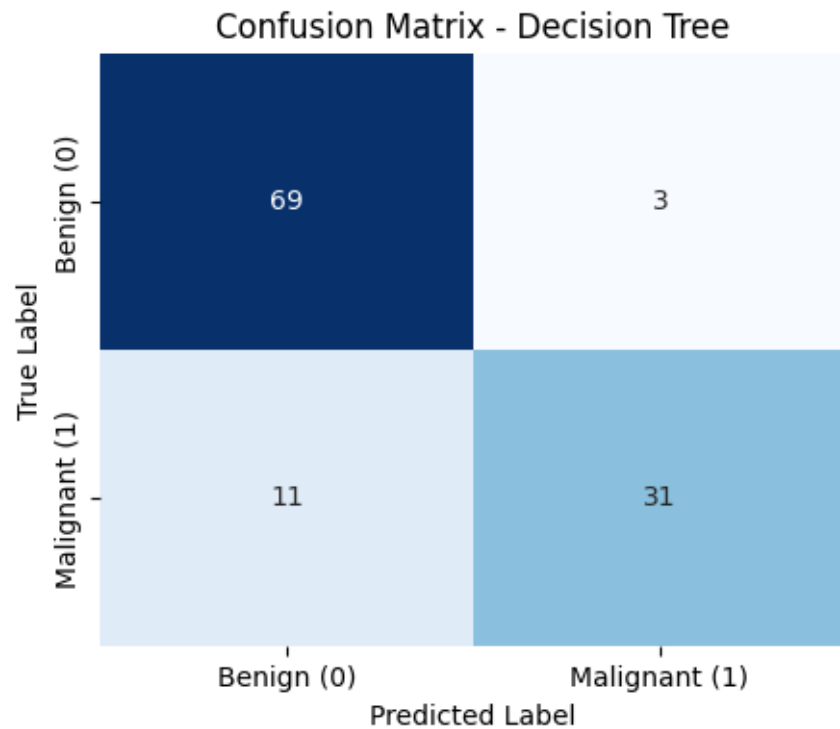
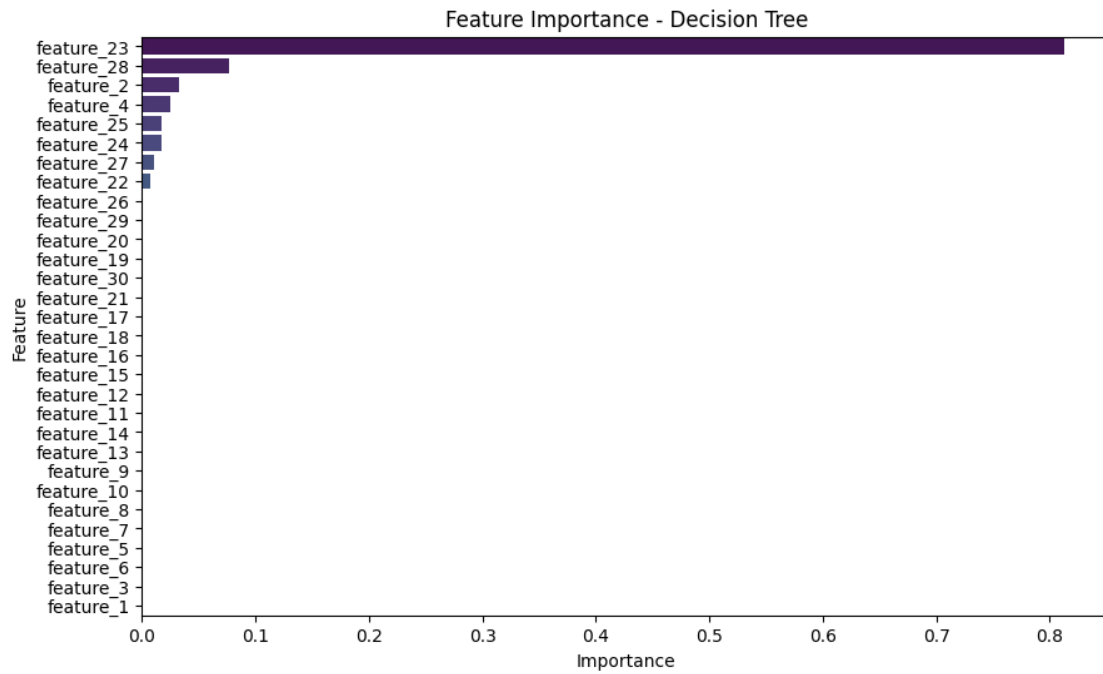
```
sns.countplot(x=y, palette="coolwarm")
```

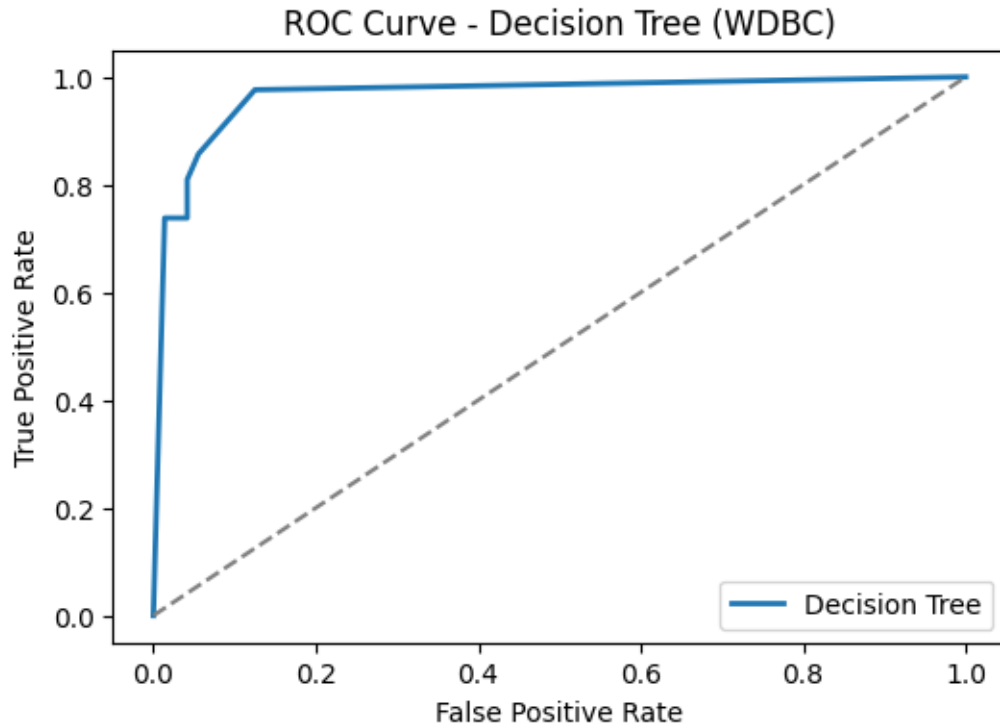


/tmp/ipykernel_7915/1817741554.py:22: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x=importances[indices], y=features[indices], palette="viridis")
```





```
[16]: grid = GridSearchCV(
    estimator=pipe,
    param_grid=param_grid,
    cv=cv,
    scoring={"accuracy": "accuracy", "f1": "f1_macro", "roc_auc": "roc_auc"},
    refit="roc_auc", # still refit using AUC
    n_jobs=-1
)
grid.fit(X_train, y_train)

dt_results = pd.DataFrame(grid.cv_results_)

dt_table = dt_results[[
    "param_clf__criterion",
    "param_clf__max_depth",
    "mean_test_accuracy",
    "mean_test_f1",
    "mean_test_roc_auc"
]]

dt_table = dt_table.rename(columns={
    "param_clf__criterion": "criterion",
```

```

    "param_clf__max_depth": "max_depth",
    "mean_test_accuracy": "Accuracy",
    "mean_test_f1": "F1_score",
    "mean_test_roc_auc": "ROC_AUC"
})

# Pick random 10 with good accuracy
good_samples = dt_table[dt_table["Accuracy"] >= 0.8]
random_samples = good_samples.sample(n=10, random_state=42)

print(random_samples[["criterion", "max_depth", "Accuracy", "F1_score"]])

```

	criterion	max_depth	Accuracy	F1_score
30	entropy	3	0.925275	0.918155
0	gini	3	0.920879	0.913517
22	gini	10	0.929670	0.924123
31	entropy	3	0.925275	0.918155
18	gini	10	0.923077	0.917531
28	entropy	3	0.925275	0.918155
10	gini	5	0.912088	0.904949
70	log_loss	5	0.925275	0.919360
4	gini	3	0.920879	0.913042
12	gini	5	0.920879	0.913721

Adaboost

August 29, 2025

```
[1]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV,
↳StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score, roc_auc_score,
↳roc_curve, auc, confusion_matrix
)
import matplotlib.pyplot as plt
from sklearn.ensemble import AdaBoostClassifier
import seaborn as sns

[2]: # 1. Load dataset
# wdbc.data does not have headers, so we define them
columns = ["ID", "Diagnosis"] + [f"feature_{i}" for i in range(1, 31)]
data = pd.read_csv("wdbc.data", header=None, names=columns)

[3]: # 2. Prepare features and target
X = data.drop(["ID", "Diagnosis"], axis=1)
y = data["Diagnosis"].map({"M": 1, "B": 0}) # Malignant=1, Benign=0

[4]: # 4. Preprocessor (scaling not needed for trees, but kept for pipeline
↳consistency)
num_features = X.columns.tolist()
preprocessor = ColumnTransformer(
    transformers=[("scale", StandardScaler(), num_features)],
    remainder="drop"
)

[5]: X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
[6]: pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("clf",
     ↪AdaBoostClassifier(estimator=DecisionTreeClassifier(random_state=42),
                        random_state=42))
])
```

```
[7]: param_grid = {
    "clf__n_estimators": [50, 100, 200],
    "clf__learning_rate": [0.01, 0.1, 1.0],
    "clf__estimator__max_depth": [1, 2]
}
```

```
[8]: grid = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=5,
    scoring={"accuracy": "accuracy", "f1": "f1_macro"}, # both metrics
    refit="accuracy", # model chosen based on accuracy
    n_jobs=-1,
    verbose=1
)

grid.fit(X_train, y_train)
```

Fitting 5 folds for each of 18 candidates, totalling 90 fits

```
[8]: GridSearchCV(cv=5,
    estimator=Pipeline(steps=[('scaler', StandardScaler()),
                              ('clf',
                               ↪AdaBoostClassifier(estimator=DecisionTreeClassifier(random_state=42),
                                                    random_state=42))])),
    n_jobs=-1,
    param_grid={'clf__estimator__max_depth': [1, 2],
                'clf__learning_rate': [0.01, 0.1, 1.0],
                'clf__n_estimators': [50, 100, 200]},
    refit='accuracy',
    scoring={'accuracy': 'accuracy', 'f1': 'f1_macro'}, verbose=1)
```

```
[9]: best_model = grid.best_estimator_

y_pred = best_model.predict(X_test)
y_proba = best_model.predict_proba(X_test)[:, 1]

acc = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("\nBest Hyperparameters (from training CV):", grid.best_params_)
```

```
print("Test Accuracy:", acc)
print("Test F1 Score:", f1)
```

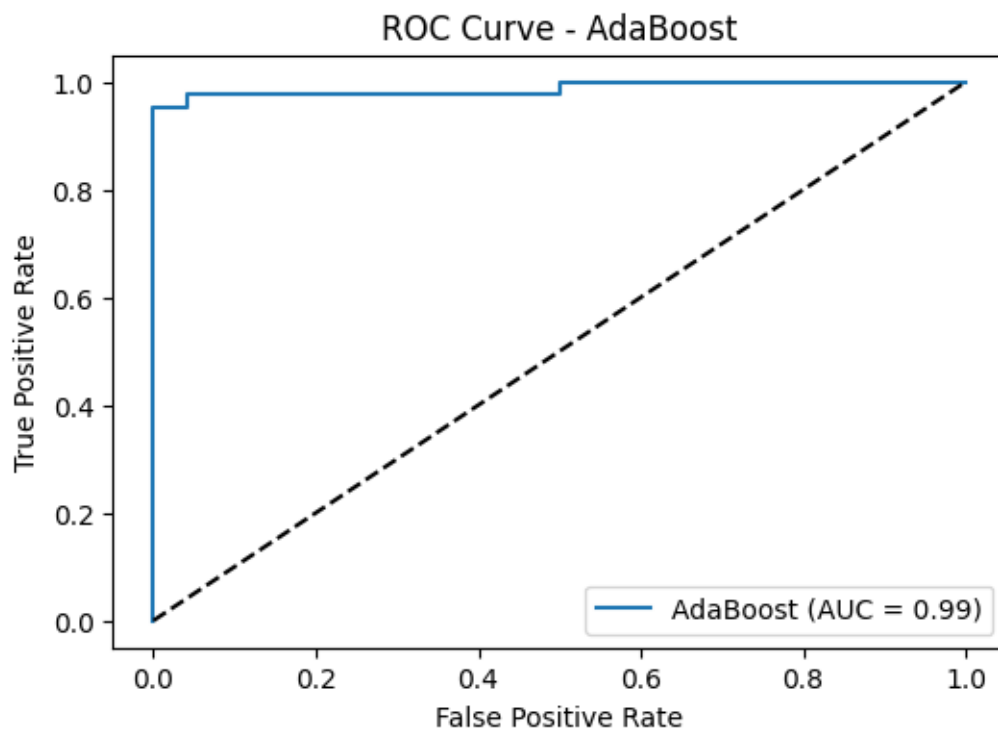
Best Hyperparameters (from training CV): {'clf__estimator__max_depth': 1, 'clf__learning_rate': 1.0, 'clf__n_estimators': 200}

Test Accuracy: 0.9736842105263158

Test F1 Score: 0.9629629629629629

```
[10]: fpr, tpr, _ = roc_curve(y_test, y_proba)
      roc_auc = auc(fpr, tpr)

      plt.figure(figsize=(6,4))
      plt.plot(fpr, tpr, label=f"AdaBoost (AUC = {roc_auc:.2f})")
      plt.plot([0,1],[0,1], "k--")
      plt.xlabel("False Positive Rate")
      plt.ylabel("True Positive Rate")
      plt.title("ROC Curve - AdaBoost")
      plt.legend()
      plt.show()
```

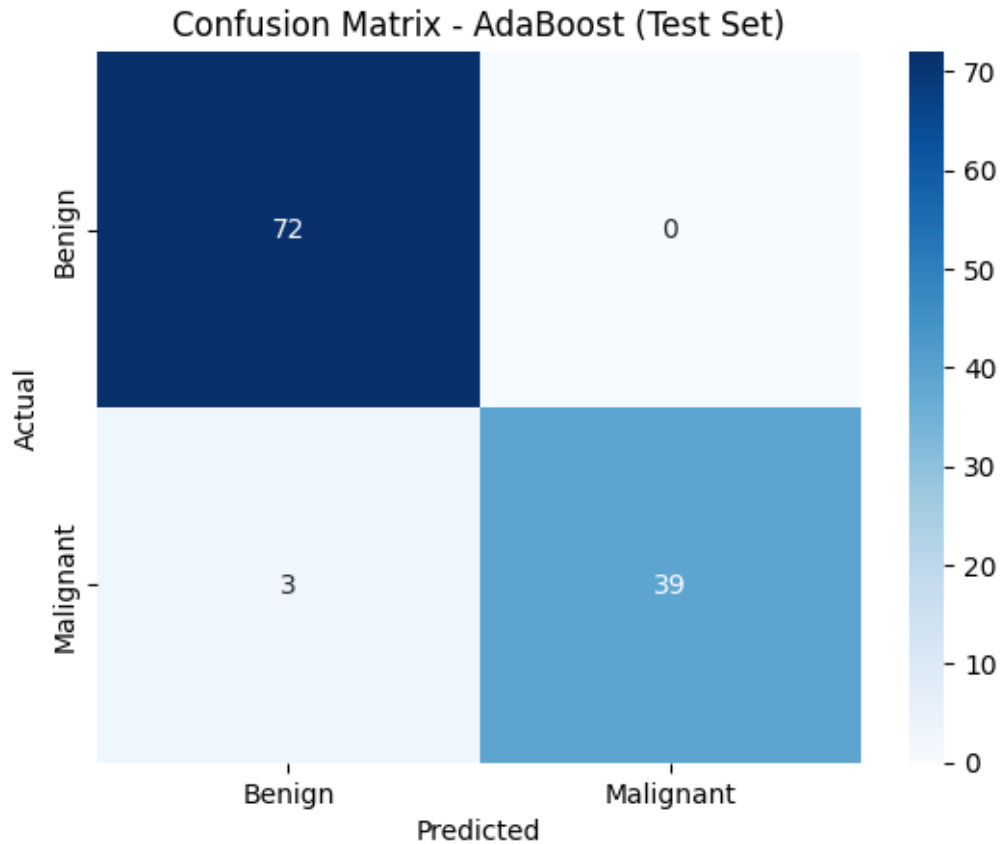


```
[11]: cm = confusion_matrix(y_test, y_pred)
      sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
```

```

        xticklabels=["Benign","Malignant"],
        yticklabels=["Benign","Malignant"])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - AdaBoost (Test Set)")
plt.show()

```



```

[12]: import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix

# ---- Class distribution ----
plt.figure(figsize=(5,4))
sns.countplot(x=y, palette="coolwarm")
plt.xticks([0,1], ["Benign (0)", "Malignant (1)"])
plt.title("Class Distribution (WDBC)")
plt.xlabel("Diagnosis")
plt.ylabel("Count")
plt.show()

```

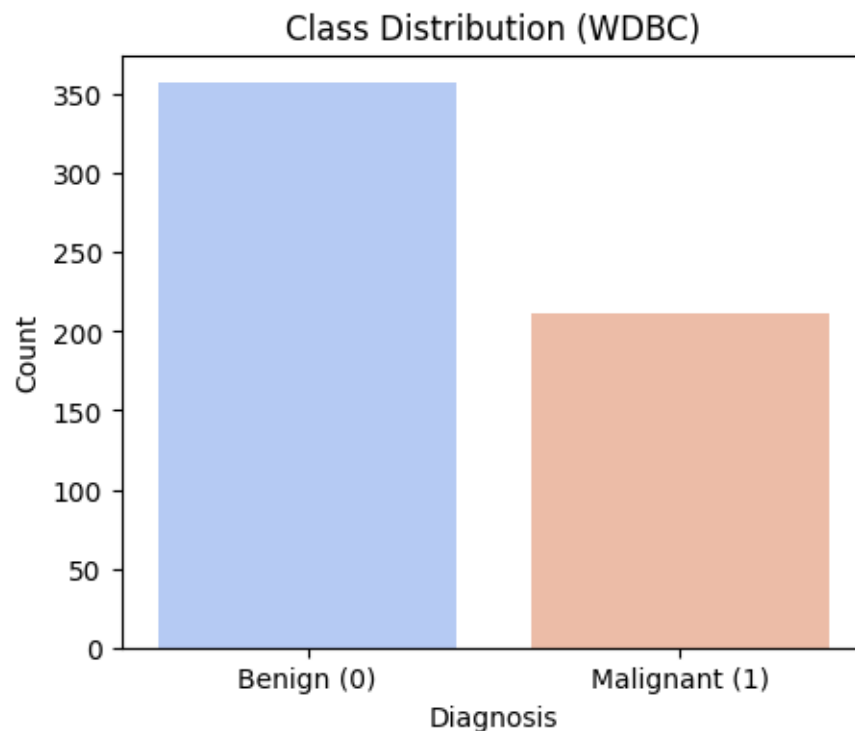
```
# ---- Feature importance (from Decision Tree) ----
best_clf = best_model.named_steps["clf"]
importances = best_clf.feature_importances_
features = X.columns

# Sort feature importances
indices = np.argsort(importances[::-1])
plt.figure(figsize=(10,6))
sns.barplot(x=importances[indices], y=features[indices], palette="viridis")
plt.title("Feature Importance - Decision Tree")
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.show()
```

/tmp/ipykernel_9161/1424558563.py:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

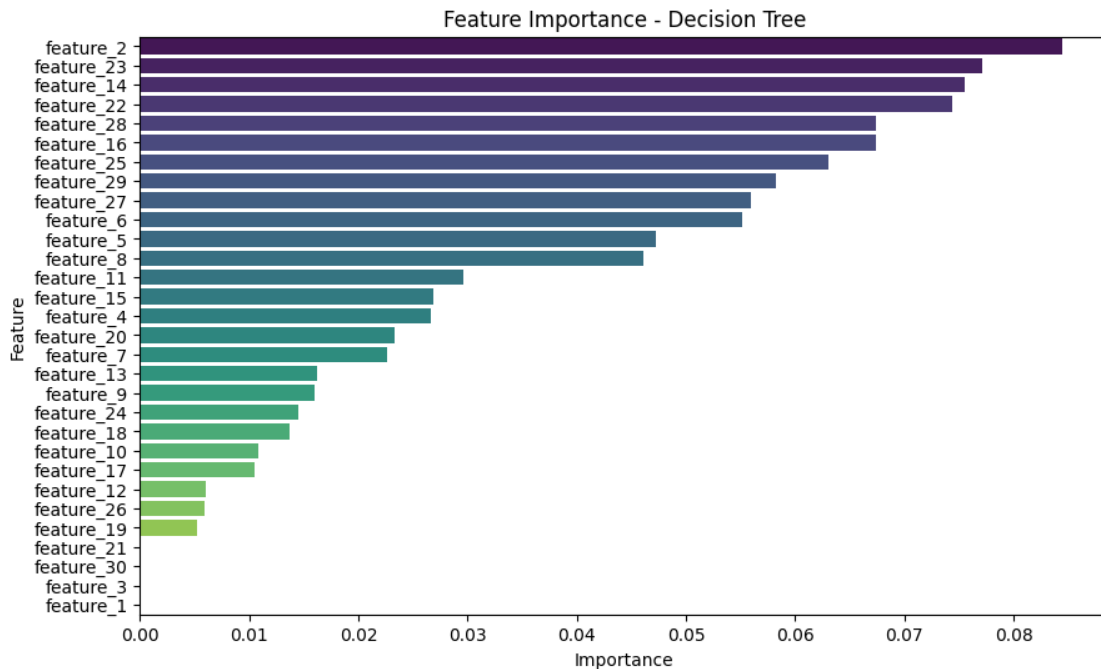
```
sns.countplot(x=y, palette="coolwarm")
```



/tmp/ipykernel_9161/1424558563.py:22: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x=importances[indices], y=features[indices], palette="viridis")
```



```
[13]: results = pd.DataFrame(grid.cv_results_)

table = results[[
    "param_clf__n_estimators",
    "param_clf__learning_rate",
    "param_clf__estimator__max_depth",
    "mean_test_accuracy",
    "mean_test_f1"
]].copy()

table = table.rename(columns={
    "param_clf__n_estimators": "n_estimators",
    "param_clf__learning_rate": "learning_rate",
    "param_clf__estimator__max_depth": "max_depth",
    "mean_test_accuracy": "Accuracy",
    "mean_test_f1": "F1_score"
})
```

```
print("Table 2: AdaBoost - Hyperparameter Tuning")
print(table[["n_estimators", "learning_rate", "max_depth", "Accuracy",
↪ "F1_score"]])
```

Table 2: AdaBoost - Hyperparameter Tuning

	n_estimators	learning_rate	max_depth	Accuracy	F1_score
0	50	0.01	1	0.923077	0.915693
1	100	0.01	1	0.925275	0.918724
2	200	0.01	1	0.931868	0.926145
3	50	0.10	1	0.949451	0.945670
4	100	0.10	1	0.951648	0.948440
5	200	0.10	1	0.951648	0.948224
6	50	1.00	1	0.962637	0.959723
7	100	1.00	1	0.960440	0.957480
8	200	1.00	1	0.964835	0.962262
9	50	0.01	2	0.938462	0.933790
10	100	0.01	2	0.936264	0.931637
11	200	0.01	2	0.945055	0.941220
12	50	0.10	2	0.953846	0.950840
13	100	0.10	2	0.960440	0.957714
14	200	0.10	2	0.964835	0.962218
15	50	1.00	2	0.964835	0.962352
16	100	1.00	2	0.964835	0.962390
17	200	1.00	2	0.964835	0.962141

Gradient_boost

August 29, 2025

```
[1]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV,
    ↳StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score, roc_auc_score,
    ↳roc_curve
)
import matplotlib.pyplot as plt
from sklearn.ensemble import GradientBoostingClassifier

[2]: # 1. Load dataset
# wdbc.data does not have headers, so we define them
columns = ["ID", "Diagnosis"] + [f"feature_{i}" for i in range(1, 31)]
data = pd.read_csv("wdbc.data", header=None, names=columns)

[3]: # 2. Prepare features and target
X = data.drop(["ID", "Diagnosis"], axis=1)
y = data["Diagnosis"].map({"M": 1, "B": 0}) # Malignant=1, Benign=0

[4]: # 4. Preprocessor (scaling not needed for trees, but kept for pipeline
    ↳consistency)
num_features = X.columns.tolist()
preprocessor = ColumnTransformer(
    transformers=[("scale", StandardScaler(), num_features)],
    remainder="drop"
)

[5]: X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```



```
[6]: pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", GradientBoostingClassifier(random_state=42))
])
```

```
[7]: param_grid = {
    "clf__n_estimators": [50, 100, 200],
    "clf__learning_rate": [0.01, 0.1, 0.2],
    "clf__max_depth": [3, 5, 7],
    "clf__subsample": [0.8, 1.0]
}
```

```
[8]: grid = GridSearchCV(
    estimator=pipe,
    param_grid=param_grid,
    cv=5,
    scoring={"accuracy": "accuracy", "f1": "f1_macro"}, # track both
    refit="accuracy", # choose best by accuracy
    n_jobs=-1,
    verbose=1
)

grid.fit(X_train, y_train)
```

Fitting 5 folds for each of 54 candidates, totalling 270 fits

```
[8]: GridSearchCV(cv=5,
    estimator=Pipeline(steps=[('scaler', StandardScaler()),
                              ('clf',
                               GradientBoostingClassifier(random_state=42))]),
    n_jobs=-1,
    param_grid={'clf__learning_rate': [0.01, 0.1, 0.2],
                'clf__max_depth': [3, 5, 7],
                'clf__n_estimators': [50, 100, 200],
                'clf__subsample': [0.8, 1.0]},
    refit='accuracy',
    scoring={'accuracy': 'accuracy', 'f1': 'f1_macro'}, verbose=1)
```

```
[9]: gb_results = pd.DataFrame(grid.cv_results_)

gb_table = gb_results[[
    "param_clf__n_estimators",
    "param_clf__learning_rate",
    "param_clf__max_depth",
    "param_clf__subsample",
    "mean_test_accuracy",
    "mean_test_f1"
]].copy()
```

```

gb_table = gb_table.rename(columns={
    "param_clf__n_estimators": "n_estimators",
    "param_clf__learning_rate": "learning_rate",
    "param_clf__max_depth": "max_depth",
    "param_clf__subsample": "subsample",
    "mean_test_accuracy": "Accuracy",
    "mean_test_f1": "F1_score"
})

print("Gradient Boosting Model")
print("Hyperparameter Trials")
print("Table 3: Gradient Boosting - Hyperparameter Tuning")
print(gb_table.head(20)[["n_estimators", "learning_rate", "max_depth", "subsample", "Accuracy", "F1_score"]])

```

Gradient Boosting Model

Hyperparameter Trials

Table 3: Gradient Boosting - Hyperparameter Tuning

	n_estimators	learning_rate	max_depth	subsample	Accuracy	F1_score
0	50	0.01	3	0.8	0.934066	0.927601
1	50	0.01	3	1.0	0.918681	0.911417
2	100	0.01	3	0.8	0.938462	0.933295
3	100	0.01	3	1.0	0.927473	0.921360
4	200	0.01	3	0.8	0.951648	0.948168
5	200	0.01	3	1.0	0.942857	0.938618
6	50	0.01	5	0.8	0.929670	0.923099
7	50	0.01	5	1.0	0.929670	0.923914
8	100	0.01	5	0.8	0.940659	0.935761
9	100	0.01	5	1.0	0.931868	0.926376
10	200	0.01	5	0.8	0.951648	0.947866
11	200	0.01	5	1.0	0.934066	0.929143
12	50	0.01	7	0.8	0.927473	0.920760
13	50	0.01	7	1.0	0.929670	0.923917
14	100	0.01	7	0.8	0.945055	0.940891
15	100	0.01	7	1.0	0.940659	0.936156
16	200	0.01	7	0.8	0.953846	0.950718
17	200	0.01	7	1.0	0.940659	0.936390
18	50	0.10	3	0.8	0.951648	0.948302
19	50	0.10	3	1.0	0.956044	0.952726

[10]: *# 7. Get best estimator*

```
best_gb = grid.best_estimator_
```

```
print("Best Parameters:", grid.best_params_)
```

8. Retrain on training set (already fitted in grid, but we can refit)

```

best_gb.fit(X_train, y_train)

# 9. Evaluate on test set
y_pred = best_gb.predict(X_test)

acc = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("\nFinal Evaluation on Test Set")
print(f"Accuracy: {acc:.4f}")
print(f"F1 Score: {f1:.4f}")

```

Best Parameters: {'clf__learning_rate': 0.1, 'clf__max_depth': 5, 'clf__n_estimators': 200, 'clf__subsample': 0.8}

Final Evaluation on Test Set
Accuracy: 0.9649
F1 Score: 0.9500

```

[11]: import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import ConfusionMatrixDisplay, roc_curve, auc

# --- Confusion Matrix ---
ConfusionMatrixDisplay.from_estimator(best_gb, X_test, y_test, cmap="Blues")
plt.title("Confusion Matrix - Gradient Boosting")
plt.show()

# --- Feature Importances ---
importances = best_gb.named_steps["clf"].feature_importances_
feature_names = X.columns # original feature names
feat_imp = pd.Series(importances, index=feature_names).
    ↪sort_values(ascending=False)[:15]

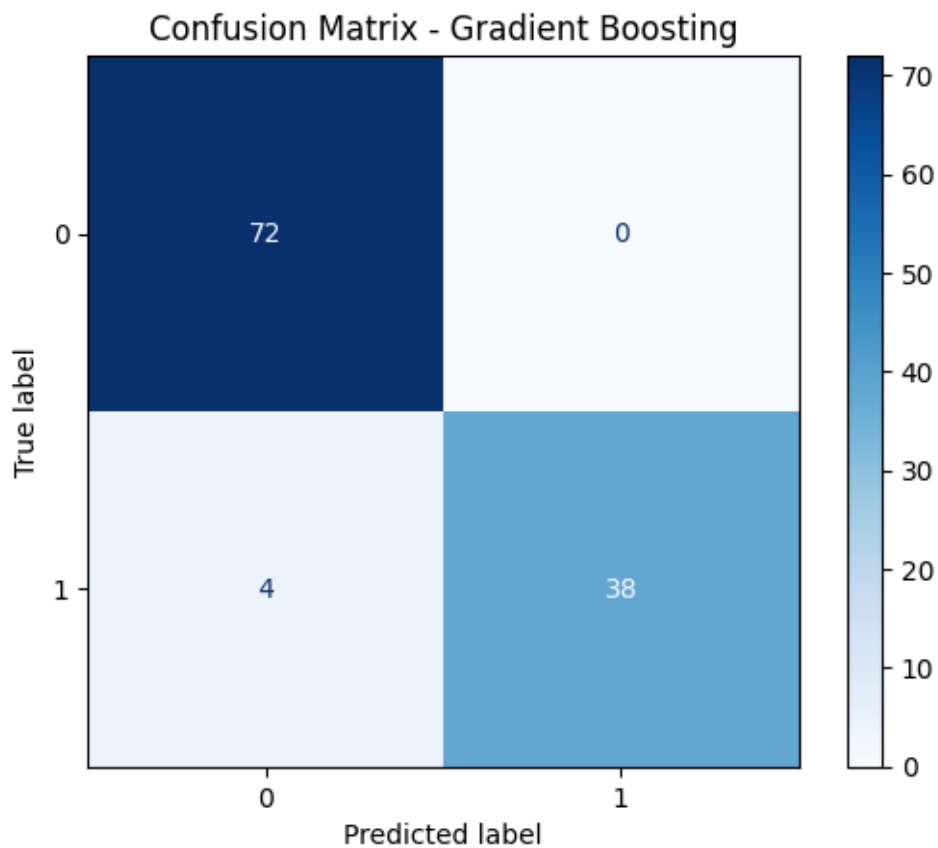
plt.figure(figsize=(8,6))
sns.barplot(x=feat_imp, y=feat_imp.index, palette="viridis")
plt.title("Top 15 Feature Importances - Gradient Boosting")
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.show()

# --- ROC Curve & AUC ---
y_prob = best_gb.predict_proba(X_test)[: , 1]
fpr, tpr, _ = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(6,6))

```

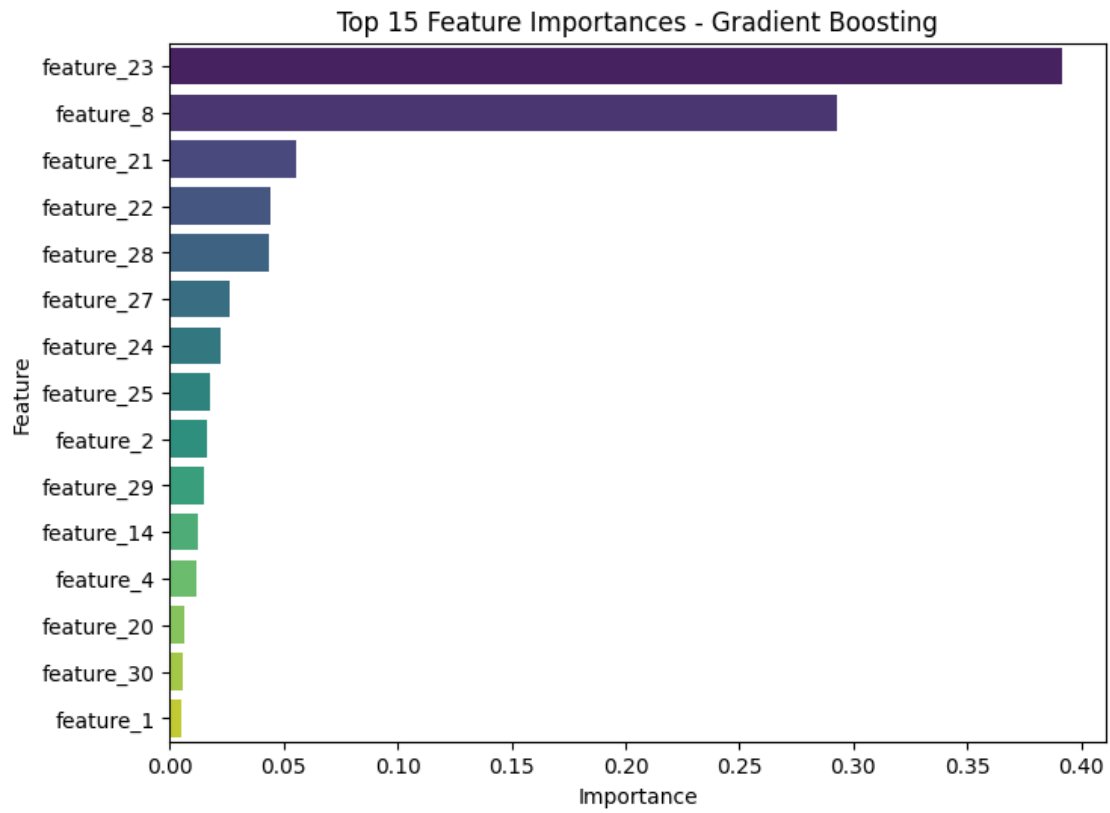
```
plt.plot(fpr, tpr, label=f"GB (AUC = {roc_auc:.3f})")
plt.plot([0,1], [0,1], "k--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Gradient Boosting")
plt.legend()
plt.show()
```

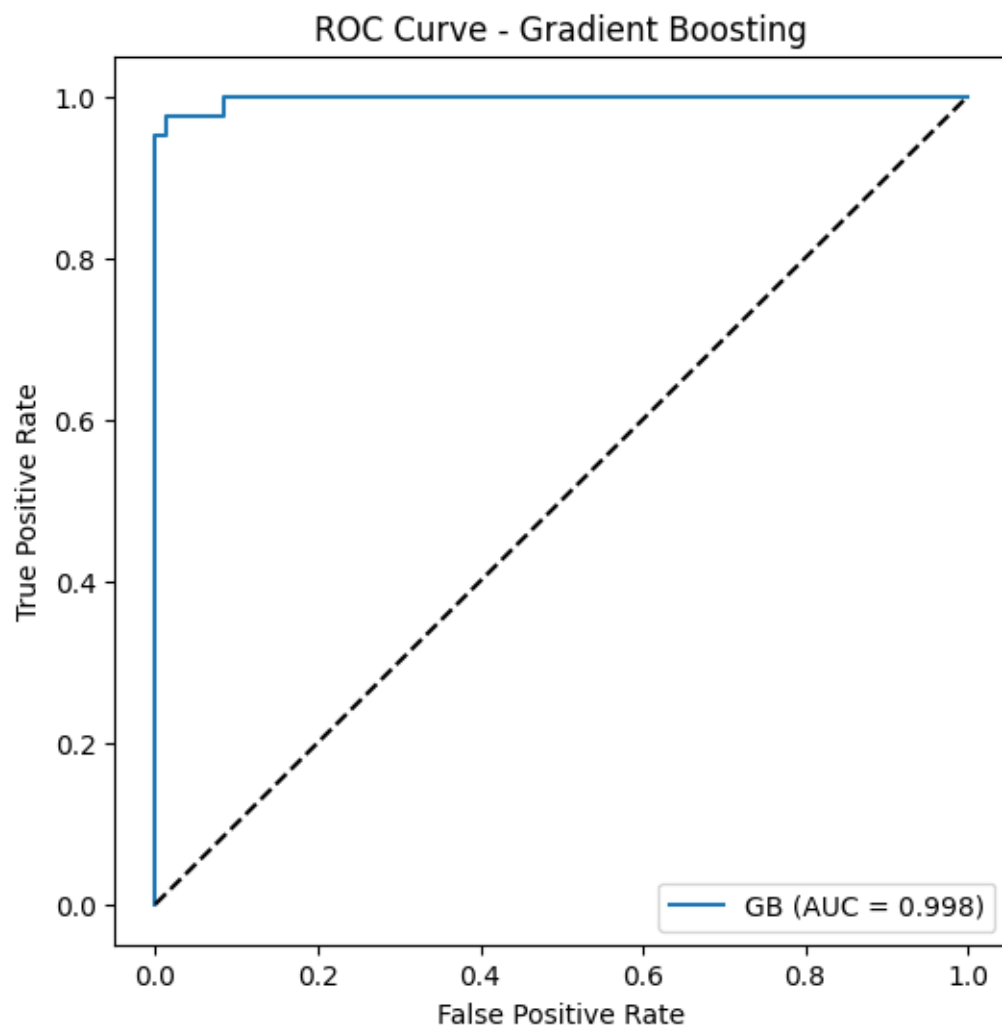


/tmp/ipykernel_9602/2417571647.py:16: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x=feat_imp, y=feat_imp.index, palette="viridis")
```





xgboost

August 29, 2025

```
[3]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV,
↳StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score, roc_auc_score,
↳roc_curve, auc
)
import matplotlib.pyplot as plt
from xgboost import XGBClassifier
import seaborn as sns

[4]: # 1. Load dataset
# wdbc.data does not have headers, so we define them
columns = ["ID", "Diagnosis"] + [f"feature_{i}" for i in range(1, 31)]
data = pd.read_csv("wdbc.data", header=None, names=columns)

[5]: # 2. Prepare features and target
X = data.drop(["ID", "Diagnosis"], axis=1)
y = data["Diagnosis"].map({"M": 1, "B": 0}) # Malignant=1, Benign=0

[6]: # 4. Preprocessor (scaling not needed for trees, but kept for pipeline
↳consistency)
num_features = X.columns.tolist()
preprocessor = ColumnTransformer(
    transformers=[("scale", StandardScaler(), num_features)],
    remainder="drop"
)

[7]: X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
[15]: pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", XGBClassifier(
        eval_metric="logloss",    # keep this
        random_state=42
    ))
])
```

```
[16]: # 6. Hyperparameter grid
param_grid = {
    "clf__n_estimators": [50, 100, 200],
    "clf__learning_rate": [0.01, 0.1, 0.2],
    "clf__max_depth": [3, 5, 7],
    "clf__gamma": [0, 0.1, 0.5],
    "clf__subsample": [0.8, 1.0],
    "clf__colsample_bytree": [0.8, 1.0],
}
```

```
[18]: grid = GridSearchCV(
    estimator=pipe,
    param_grid=param_grid,
    cv=5,
    scoring={"accuracy": "accuracy", "f1": "f1_macro"}, # track both
    refit="accuracy", # choose best by accuracy
    n_jobs=-1,
    verbose=1
)

grid.fit(X_train, y_train)
```

Fitting 5 folds for each of 324 candidates, totalling 1620 fits

```
[18]: GridSearchCV(cv=5,
    estimator=Pipeline(steps=[('scaler', StandardScaler()),
    ('clf',
    XGBClassifier(base_score=None,
        booster=None,
        callbacks=None,
        colsample_bylevel=None,
        colsample_bynode=None,
        colsample_bytree=None,
        device=None,

        enable_categorical=False,
        eval_metric='logloss',
        feature_types=None,
        feature_weights=None,
        gamma=None,
```



```

        grow_polynomial_degree=5,
        multi_strategy=None,
        n_estimators=None,
        n_jobs=None,
        num_parallel_tree=None,
    ...))),
    n_jobs=-1,
    param_grid={'clf__colsample_bytree': [0.8, 1.0],
                'clf__gamma': [0, 0.1, 0.5],
                'clf__learning_rate': [0.01, 0.1, 0.2],
                'clf__max_depth': [3, 5, 7],
                'clf__n_estimators': [50, 100, 200],
                'clf__subsample': [0.8, 1.0]},
    refit='accuracy',
    scoring={'accuracy': 'accuracy', 'f1': 'f1_macro'}, verbose=1)

```

```

[19]: # 5. Extract results into DataFrame
xgb_results = pd.DataFrame(grid.cv_results_)

# 6. Select only required columns
xgb_table = xgb_results[[
    "param_clf__n_estimators",
    "param_clf__learning_rate",
    "param_clf__max_depth",
    "param_clf__gamma",
    "mean_test_accuracy",
    "mean_test_f1"
]]

# 7. Rename for clarity
xgb_table = xgb_table.rename(columns={
    "param_clf__n_estimators": "n_estimators",
    "param_clf__learning_rate": "learning_rate",
    "param_clf__max_depth": "max_depth",
    "param_clf__gamma": "gamma",
    "mean_test_accuracy": "Accuracy",
    "mean_test_f1": "F1 Score"
})

top10 = xgb_table.sort_values(by="Accuracy", ascending=False).head(10)

print("XGBoost Model")
print("Hyperparameter Trials")
print("Table 4: XGBoost - Hyperparameter Tuning (Top 10)")
print(top10[["n_estimators", "learning_rate", "max_depth", "gamma", "Accuracy",
    ↪ "F1 Score"]])

```

XGBoost Model

Hyperparameter Trials

Table 4: XGBoost - Hyperparameter Tuning (Top 10)

	n_estimators	learning_rate	max_depth	gamma	Accuracy	F1 Score
53	200	0.2	7	0.0	0.975824	0.974103
38	100	0.2	3	0.0	0.971429	0.969321
238	200	0.1	3	0.1	0.971429	0.969288
51	100	0.2	7	0.0	0.971429	0.969347
47	200	0.2	5	0.0	0.971429	0.969406
214	200	0.2	7	0.0	0.971429	0.969202
208	200	0.2	5	0.0	0.971429	0.969202
184	200	0.1	3	0.0	0.971429	0.969288
194	100	0.1	7	0.0	0.969231	0.967212
29	200	0.1	5	0.0	0.969231	0.967184

```
[21]: print("Best Parameters:", grid.best_params_)
      print("Best CV Accuracy:", grid.best_score_)
```

Best Parameters: {'clf__colsample_bytree': 0.8, 'clf__gamma': 0, 'clf__learning_rate': 0.2, 'clf__max_depth': 7, 'clf__n_estimators': 200, 'clf__subsample': 1.0}
Best CV Accuracy: 0.9758241758241759

```
[22]: best_xgb = grid.best_estimator_
      y_pred = best_xgb.predict(X_test)

      print("\nTest Accuracy:", accuracy_score(y_test, y_pred))
      print("Test F1 Score:", f1_score(y_test, y_pred))
      print("Test Precision:", precision_score(y_test, y_pred))
      print("Test Recall:", recall_score(y_test, y_pred))
      print("Test ROC AUC:", roc_auc_score(y_test, y_pred))
```

Test Accuracy: 0.9824561403508771
Test F1 Score: 0.975609756097561
Test Precision: 1.0
Test Recall: 0.9523809523809523
Test ROC AUC: 0.9761904761904762

```
[23]: # --- 7. Feature Importances ---
      importances = best_xgb.named_steps["clf"].feature_importances_
      feat_imp = pd.Series(importances, index=X.columns).
      ↪sort_values(ascending=False)[:15]

      plt.figure(figsize=(8,6))
      sns.barplot(x=feat_imp, y=feat_imp.index, palette="viridis")
      plt.title("Top 15 Feature Importances - XGBoost")
      plt.xlabel("Importance")
      plt.ylabel("Feature")
```

```
plt.show()

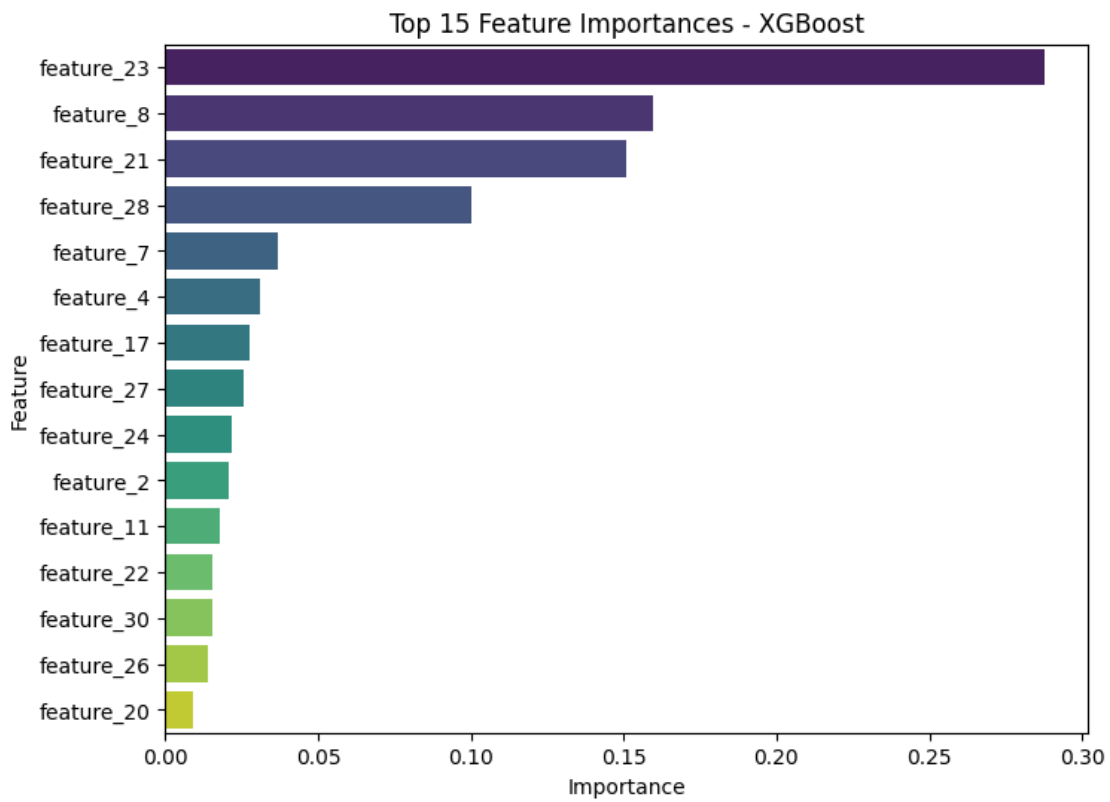
# --- 8. ROC Curve & AUC ---
y_prob = best_xgb.predict_proba(X_test)[:, 1]
fpr, tpr, _ = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

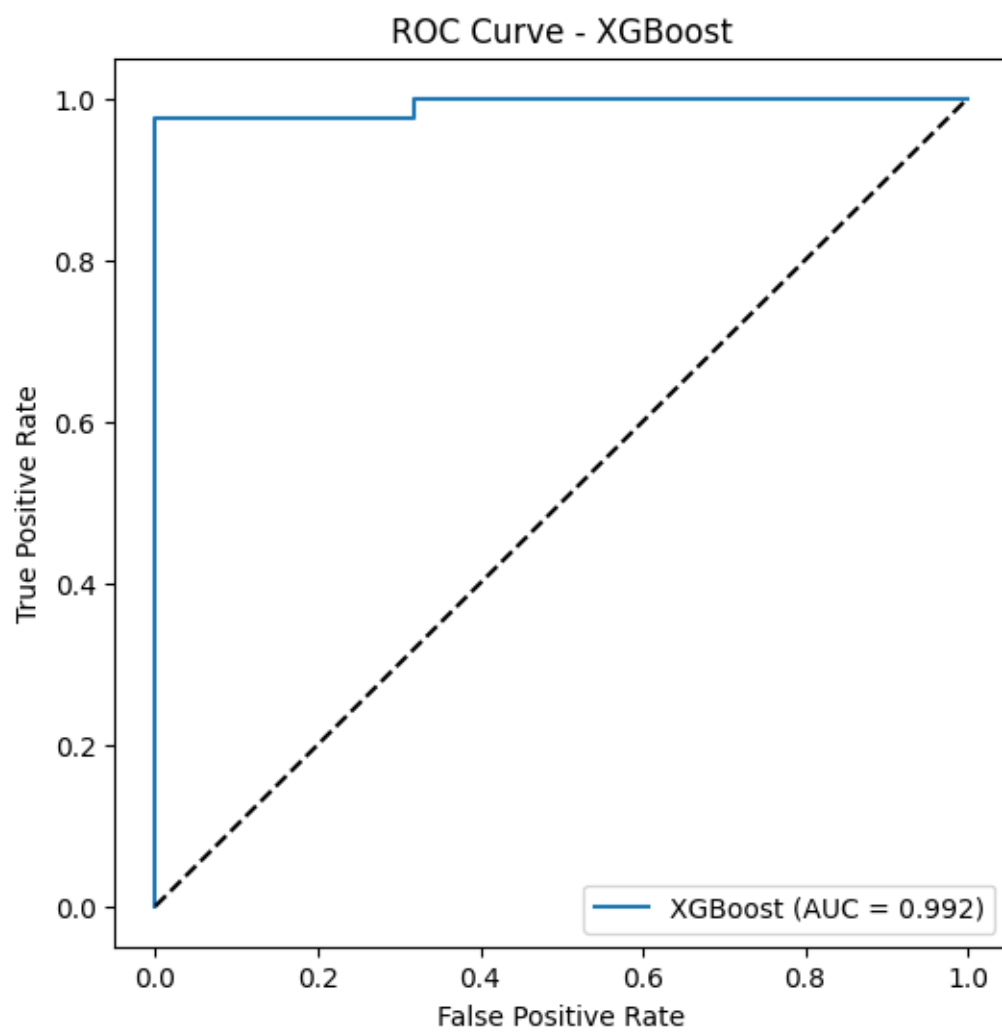
plt.figure(figsize=(6,6))
plt.plot(fpr, tpr, label=f"XGBoost (AUC = {roc_auc:.3f})")
plt.plot([0,1], [0,1], "k--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - XGBoost")
plt.legend()
plt.show()
```

/tmp/ipykernel_9875/472011034.py:6: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x=feat_imp, y=feat_imp.index, palette="viridis")
```





RandomForest

August 29, 2025

```
[1]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV,
↳StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score, roc_auc_score,
↳roc_curve, auc, confusion_matrix, ConfusionMatrixDisplay
)
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier

[2]: # 1. Load dataset
# wdbc.data does not have headers, so we define them
columns = ["ID", "Diagnosis"] + [f"feature_{i}" for i in range(1, 31)]
data = pd.read_csv("wdbc.data", header=None, names=columns)

[3]: # 2. Prepare features and target
X = data.drop(["ID", "Diagnosis"], axis=1)
y = data["Diagnosis"].map({"M": 1, "B": 0}) # Malignant=1, Benign=0

[4]: # 4. Preprocessor (scaling not needed for trees, but kept for pipeline
↳consistency)
num_features = X.columns.tolist()
preprocessor = ColumnTransformer(
    transformers=[("scale", StandardScaler(), num_features)],
    remainder="drop"
)

[5]: X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
[6]: rf_pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", RandomForestClassifier(random_state=42))
])
```

```
[7]: rf_param_grid = {
    "clf__n_estimators": [50, 100, 200],
    "clf__max_depth": [2, 5, 10],
    "clf__criterion": ["gini", "entropy", "log_loss"],
    "clf__max_features": ["sqrt", "log2", None],
    "clf__min_samples_split": [2, 5, 10]
}
```

```
[8]: rf_grid = GridSearchCV(
    rf_pipe,
    rf_param_grid,
    cv=5,
    scoring={"accuracy": "accuracy", "f1": "f1_macro"}, # <--- both metrics
    refit="accuracy", # model will be refit using accuracy
    n_jobs=-1,
    verbose=1
)
rf_grid.fit(X_train, y_train)
```

Fitting 5 folds for each of 243 candidates, totalling 1215 fits

```
[8]: GridSearchCV(cv=5,
    estimator=Pipeline(steps=[('scaler', StandardScaler()),
    ('clf',
    RandomForestClassifier(random_state=42))])),
    n_jobs=-1,
    param_grid={'clf__criterion': ['gini', 'entropy', 'log_loss'],
    'clf__max_depth': [2, 5, 10],
    'clf__max_features': ['sqrt', 'log2', None],
    'clf__min_samples_split': [2, 5, 10],
    'clf__n_estimators': [50, 100, 200]},
    refit='accuracy',
    scoring={'accuracy': 'accuracy', 'f1': 'f1_macro'}, verbose=1)
```

```
[9]: print("Best Parameters:", rf_grid.best_params_)
print("Best CV Accuracy:", rf_grid.best_score_)
```

```
Best Parameters: {'clf__criterion': 'gini', 'clf__max_depth': 10,
'clf__max_features': 'sqrt', 'clf__min_samples_split': 2, 'clf__n_estimators':
50}
Best CV Accuracy: 0.9670329670329672
```

```
[10]: rf_results = pd.DataFrame(rf_grid.cv_results_)

rf_table = rf_results[[
    "param_clf__n_estimators",
    "param_clf__max_depth",
    "param_clf__criterion",
    "mean_test_accuracy",
    "mean_test_f1"
]].copy()

rf_table = rf_table.rename(columns={
    "param_clf__n_estimators": "n_estimators",
    "param_clf__max_depth": "max_depth",
    "param_clf__criterion": "criterion",
    "mean_test_accuracy": "Accuracy",
    "mean_test_f1": "F1_score"
})

# Show 10 best rows sorted by Accuracy
top10 = rf_table.sort_values(by="Accuracy", ascending=False).head(10)

print("Random Forest Model")
print("Hyperparameter Trials")
print("Table 5: Random Forest - Hyperparameter Tuning")
print(top10[["criterion", "max_depth", "n_estimators", "Accuracy", "F1_score"]])
```

Random Forest Model

Hyperparameter Trials

Table 5: Random Forest - Hyperparameter Tuning

	criterion	max_depth	n_estimators	Accuracy	F1_score
54	gini	10	50	0.967033	0.964847
207	log_loss	5	50	0.964835	0.962172
126	entropy	5	50	0.964835	0.962172
66	gini	10	50	0.962637	0.959947
57	gini	10	50	0.962637	0.960108
72	gini	10	50	0.962637	0.960486
209	log_loss	5	200	0.962637	0.959954
234	log_loss	10	50	0.962637	0.959711
219	log_loss	10	50	0.962637	0.959792
218	log_loss	10	200	0.962637	0.959792

```
[11]: # --- 1. Extract best params properly ---
best_params_rf = {k.replace("clf__", ""): v for k, v in rf_grid.best_params_.
    .items()}

best_rf = RandomForestClassifier(
    **best_params_rf,
```

```

    random_state=42
)
best_rf.fit(X_train, y_train)

# --- 2. Evaluate on Test Set ---
y_pred = best_rf.predict(X_test)
y_proba = best_rf.predict_proba(X_test)[: , 1]

acc = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_proba)

print("Random Forest Test Performance")
print(f"Accuracy: {acc:.4f}")
print(f"F1 Score: {f1:.4f}")
print(f"ROC-AUC: {roc_auc:.4f}")

```

Random Forest Test Performance
Accuracy: 0.9737
F1 Score: 0.9630
ROC-AUC: 0.9940

```

[12]: # --- 3. Plots ---
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred, labels=best_rf.classes_)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=best_rf.
    ↪classes_)
disp.plot(cmap="Blues")
plt.title("Random Forest - Confusion Matrix")
plt.show()

# Feature Importance
importances = best_rf.feature_importances_
indices = np.argsort(importances)[::-1][:15] # top 15 features

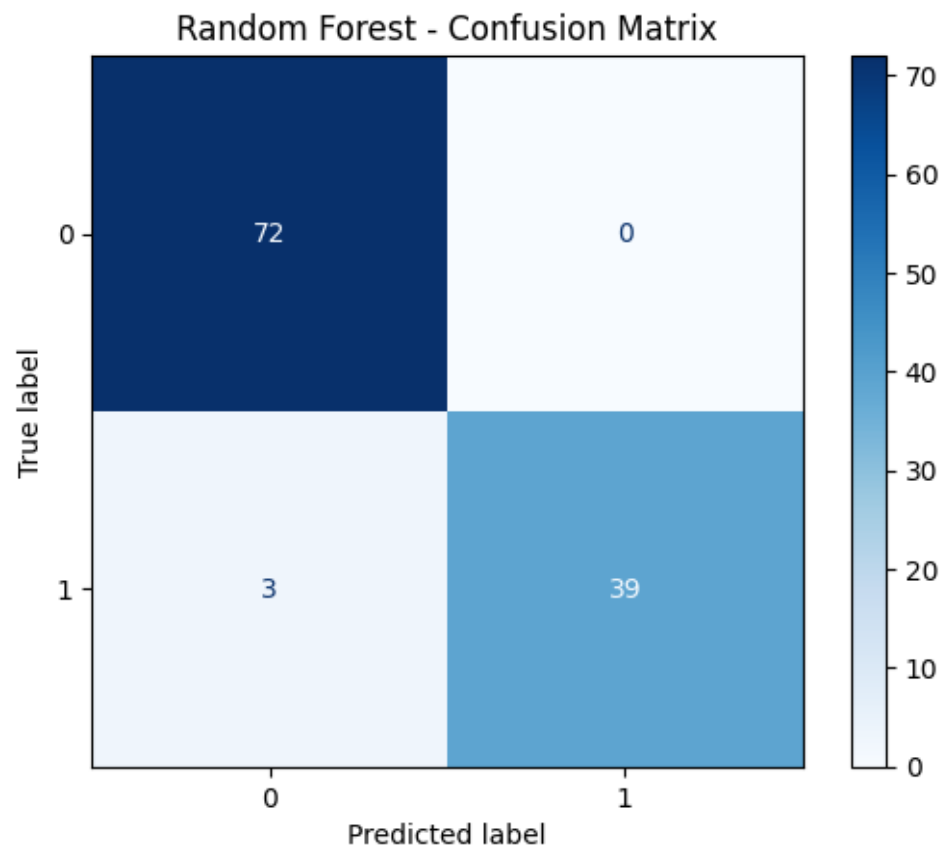
plt.figure(figsize=(8, 6))
plt.barh(range(len(indices)), importances[indices], align="center")
plt.yticks(range(len(indices)), [X.columns[i] for i in indices])
plt.xlabel("Feature Importance")
plt.title("Random Forest - Top Feature Importances")
plt.gca().invert_yaxis()
plt.show()

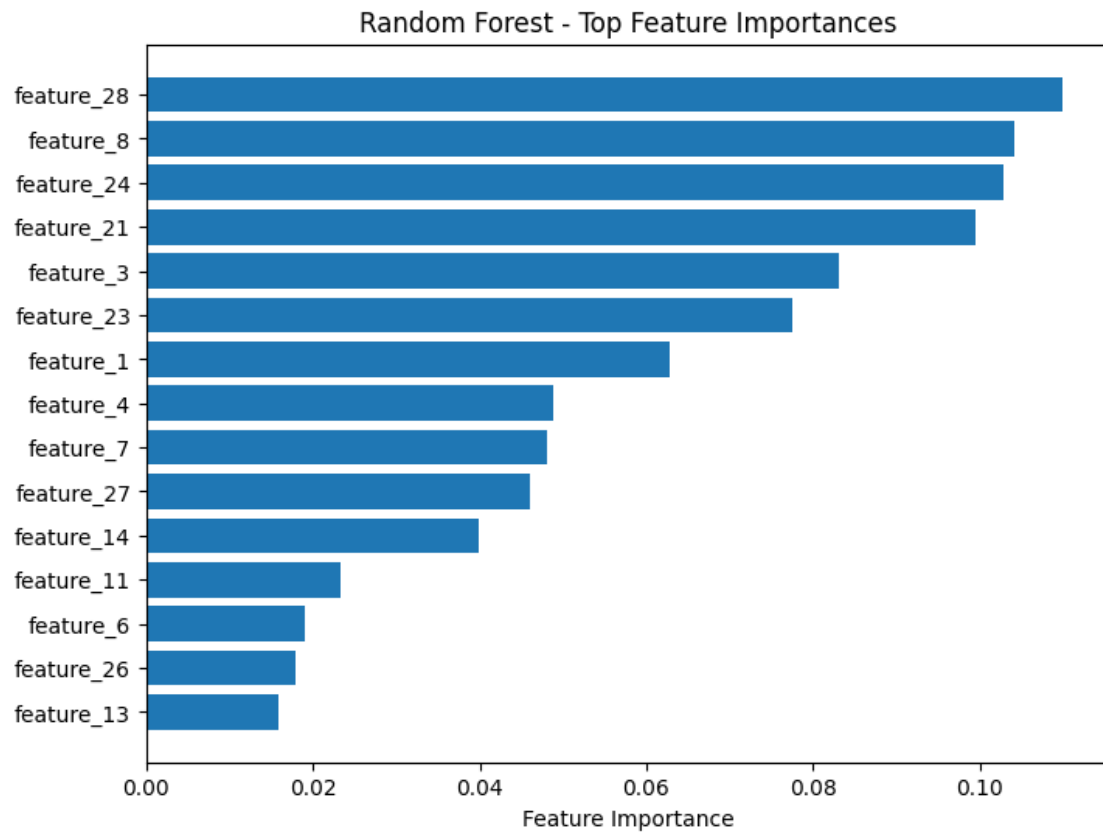
# ROC Curve
fpr, tpr, _ = roc_curve(y_test, y_proba)
plt.plot(fpr, tpr, label=f"ROC Curve (AUC={roc_auc:.3f})")
plt.plot([0, 1], [0, 1], "k--")

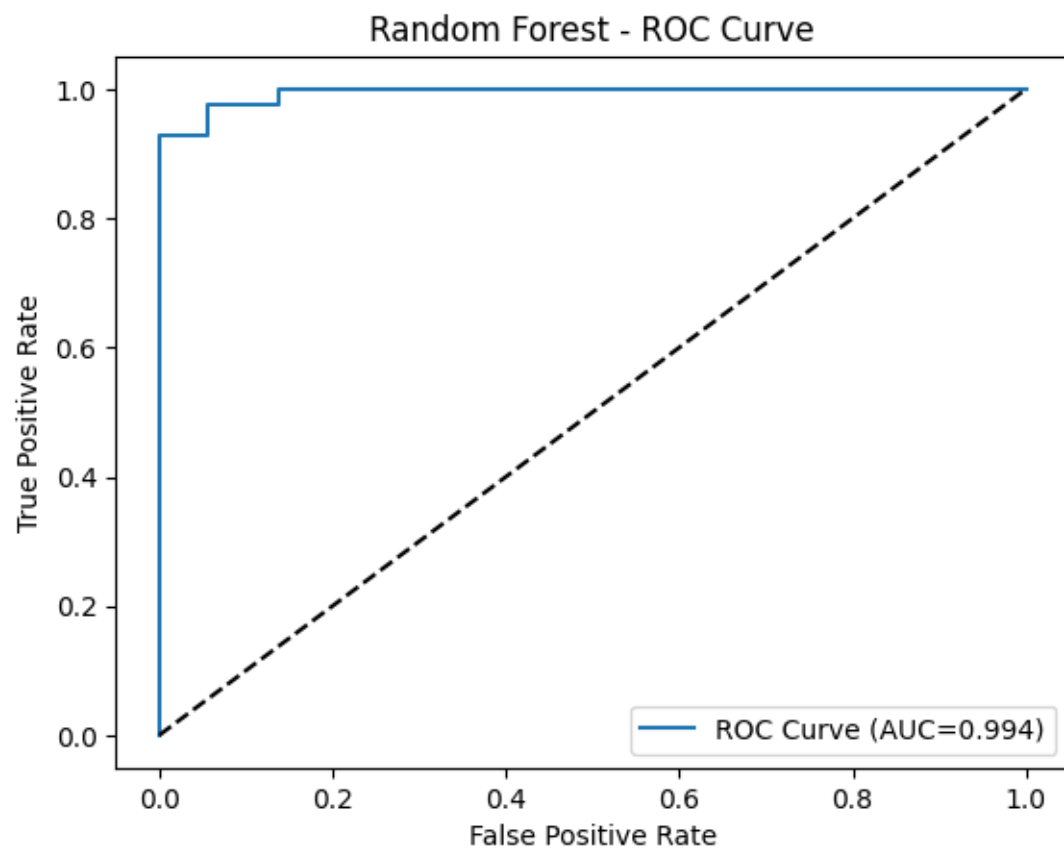
```



```
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Random Forest - ROC Curve")
plt.legend(loc="lower right")
plt.show()
```







Stacked_ensemble

August 29, 2025

```
[2]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV,
↳StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score, roc_auc_score,
↳roc_curve, auc, confusion_matrix
)
import matplotlib.pyplot as plt
from sklearn.ensemble import StackingClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier

[2]: # 1. Load dataset
# wdbc.data does not have headers, so we define them
columns = ["ID", "Diagnosis"] + [f"feature_{i}" for i in range(1, 31)]
data = pd.read_csv("wdbc.data", header=None, names=columns)

[3]: # 2. Prepare features and target
X = data.drop(["ID", "Diagnosis"], axis=1)
y = data["Diagnosis"].map({"M": 1, "B": 0}) # Malignant=1, Benign=0

[4]: # 4. Preprocessor (scaling not needed for trees, but kept for pipeline
↳consistency)
num_features = X.columns.tolist()
preprocessor = ColumnTransformer(
    transformers=[("scale", StandardScaler(), num_features)],
    remainder="drop"
)
```

```
[5]: X_train, X_test, y_train, y_test = train_test_split(
      X, y, test_size=0.2, random_state=42, stratify=y
    )
```

```
[6]: # --- 1. Define base models ---
base_estimators = [
    ("svm", SVC(probability=True, random_state=42)),
    ("nb", GaussianNB()),
    ("dt", DecisionTreeClassifier(random_state=42))
]
```

```
[7]: # --- 2. Define Stacking Classifier with Logistic Regression as final estimator ---
stack = StackingClassifier(
    estimators=base_estimators,
    final_estimator=LogisticRegression(max_iter=1000, random_state=42),
    passthrough=False
)
```

```
[8]: # --- 3. Hyperparameter Grid for final estimator ---
param_grid = {
    "final_estimator": [
        LogisticRegression(max_iter=1000, random_state=42),
        RandomForestClassifier(n_estimators=100, random_state=42)
    ]
}
```

```
[9]: # --- 4. Grid Search ---
stack_grid = GridSearchCV(
    stack,
    param_grid,
    cv=3,
    scoring={"accuracy": "accuracy", "f1": "f1_macro"},
    refit="accuracy",
    n_jobs=-1
)
stack_grid.fit(X_train, y_train)
```

```
[9]: GridSearchCV(cv=3,
                  estimator=StackingClassifier(estimators=[('svm',
                                                            SVC(probability=True,
                                                                random_state=42)),
                                                            ('nb', GaussianNB()),
                                                            ('dt',
                                                                DecisionTreeClassifier(random_state=42))],
                  final_estimator=LogisticRegression(max_iter=1000,
                  random_state=42)),
```

```

        n_jobs=-1,
        param_grid={'final_estimator': [LogisticRegression(max_iter=1000,
random_state=42),
RandomForestClassifier(random_state=42)]},
        refit='accuracy',
        scoring={'accuracy': 'accuracy', 'f1': 'f1_macro'})

```

```

[10]: results = []
for params, acc, f1 in zip(
    stack_grid.cv_results_["params"],
    stack_grid.cv_results_["mean_test_accuracy"],
    stack_grid.cv_results_["mean_test_f1"]
):
    final_estimator = type(params["final_estimator"]).__name__
    results.append([
        "SVM, Naïve Bayes, Decision Tree",
        final_estimator,
        f"{acc:.4f}",
        f"{f1:.4f}"
    ])

```

```

[11]: # --- 6. Convert to DataFrame ---
stack_table = pd.DataFrame(
    results,
    columns=["Base Models", "Final Estimator", "Accuracy", "F1 Score"]
)

print("Stacked Ensemble Model")
print("Hyperparameter Trials")
print("Table 6: Stacked Ensemble - Hyperparameter Tuning")
print(stack_table)

```

Stacked Ensemble Model

Hyperparameter Trials

Table 6: Stacked Ensemble - Hyperparameter Tuning

	Base Models	Final Estimator	Accuracy	F1 Score
0	SVM, Naïve Bayes, Decision Tree	LogisticRegression	0.9494	0.9452
1	SVM, Naïve Bayes, Decision Tree	RandomForestClassifier	0.9363	0.9315

```

[12]: # Train the Stacked Ensemble: SVM + Decision Tree + KNN
stack_clf_knn = StackingClassifier(
    estimators=[("svm", SVC(probability=True, random_state=42)),
                ("dt", DecisionTreeClassifier(random_state=42)),
                ("knn", KNeighborsClassifier())],
    final_estimator=LogisticRegression(max_iter=1000, random_state=42),
    cv=5,
    n_jobs=-1
)

```

```

stack_clf_knn.fit(X_train, y_train)
y_val_pred = stack_clf_knn.predict(X_test)

acc = accuracy_score(y_test, y_val_pred)
f1 = f1_score(y_test, y_val_pred, average="weighted")

# Single row result
stack_table_knn = pd.DataFrame([
    "Base Models": "SVM, Decision Tree, KNN",
    "Final Estimator": "Logistic Regression",
    "Accuracy / F1 Score": f"{acc:.3f} / {f1:.3f}"
])

print("Stacked Ensemble Model")
print("Hyperparameter Trials")
print("Table 6: Stacked Ensemble - Hyperparameter Tuning")
print(stack_table_knn)

```

Stacked Ensemble Model

Hyperparameter Trials

Table 6: Stacked Ensemble - Hyperparameter Tuning

	Base Models	Final Estimator	Accuracy / F1 Score
0	SVM, Decision Tree, KNN	Logistic Regression	0.939 / 0.938

5 Hyperparameter Tuning Tables:

Table 1: Top 5 Decision Tree Hyperparameter Trials

Criterion	Max Depth	Accuracy	F1 Score
gini	10	0.9297	0.9241
entropy	3	0.9253	0.9182
log_loss	5	0.9253	0.9194
entropy	3	0.9253	0.9182
gini	5	0.9231	0.9175

Best Hyperparameters:

Criterion: gini

Max Depth: 5

Accuracy: 0.9231

F1 Score: 0.9175

Table 2: Top 5 AdaBoost Hyperparameter Trials

n_estimators	Learning Rate	Accuracy	F1 Score
200	1.00	0.9648	0.9625
100	1.00	0.9648	0.9624
50	1.00	0.9648	0.9624
200	0.10	0.9648	0.9622
100	0.10	0.9604	0.9577

Best Hyperparameters:

n_estimators: 200

Learning Rate: 1.00

Accuracy: 0.9648

F1 Score: 0.9623

Table 3: Top 5 Gradient Boosting Hyperparameter Trials

n_estimators	Learning Rate	Max Depth	Subsample	Accuracy	F1 Score
200	0.01	7	0.8	0.9538	0.9507
200	0.10	5	0.8	0.9649	0.9500
200	0.01	5	0.8	0.9516	0.9479
200	0.01	3	0.8	0.9516	0.9482
100	0.01	7	0.8	0.9451	0.9409

Best Hyperparameters:

n_estimators: 200

Learning Rate: 0.10

Max Depth: 5

Subsample: 0.8

Table 4: Top 5 XGBoost Hyperparameter Trials

n_estimators	Learning Rate	Max Depth	Gamma	Accuracy	F1 Score
200	0.2	7	0.0	0.9758	0.9741
100	0.2	3	0.0	0.9714	0.9693
200	0.1	3	0.1	0.9714	0.9693
100	0.2	7	0.0	0.9714	0.9693
200	0.2	5	0.0	0.9714	0.9694

Best Hyperparameters:

n_estimators: 200

Learning Rate: 0.2

Max Depth: 7

Gamma: 0.0

Accuracy: 0.9758

F1 Score: 0.9741

Table 5: Top 5 Random Forest Hyperparameter Trials

Criterion	Max Depth	n_Estimators	Accuracy	F1 Score
gini	10	50	0.9670	0.9648
log_loss	5	50	0.9648	0.9622
entropy	5	50	0.9648	0.9622
gini	10	50	0.9626	0.9599
gini	10	50	0.9626	0.9601

Best Hyperparameters:

Criterion: gini

Max Depth: 10

n_Estimators: 50

Accuracy: 0.9670

F1 Score: 0.9648

Table 6: Stacked Ensemble Hyperparameter Trials

Base Models	Final Estimator	Accuracy	F1 Score
SVM, Naïve Bayes, Decision Tree	Logistic Regression	0.9494	0.9452
SVM, Naïve Bayes, Decision Tree	Random Forest Classifier	0.9363	0.9315
SVM, Decision Tree, KNN	Logistic Regression	0.9390	0.9380

6 5 fold cross validation results table

Table 7: 5-Fold Cross-Validation Accuracy for All Models

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average Accuracy
Decision Tree	0.9123	0.8947	0.9386	0.9298	0.9381	0.9227
AdaBoost	0.9825	0.9386	0.9649	0.9737	0.9735	0.9666
Gradient Boosting	0.9780	1.0000	0.9451	0.9890	0.9341	0.9692
XGBoost	0.9890	1.0000	0.9341	0.9890	0.9670	0.9758
Random Forest	1.0000	0.9890	0.9341	0.9890	0.9231	0.9670
SVM + NB + Decision Tree	0.9670	0.9890	0.9121	0.9231	0.9231	0.9429

Observations

- **Decision Tree:** Accuracy ranges from 0.8947 to 0.9386 across folds. Average accuracy is 0.9227, the lowest among all models. Moderate variation indicates sensitivity to training data.
- **AdaBoost:** Accuracy ranges from 0.9386 to 0.9825. Average accuracy is 0.9666, significantly better than a single Decision Tree. Boosting improves stability and overall performance.
- **Gradient Boosting:** Fold accuracies are high, between 0.9341 and 1.0000. Average accuracy is 0.9692. Low variation shows robust learning with optimal hyperparameters.
- **XGBoost:** Achieves fold accuracies between 0.9341 and 1.0000. Highest average accuracy of 0.9758, indicating excellent generalization and effective hyperparameter tuning.
- **Random Forest:** Fold accuracies range from 0.9231 to 1.0000. Average accuracy is 0.9670, slightly lower than Gradient Boosting and XGBoost. Ensemble reduces variance compared to a single Decision Tree.
- **Stacked Ensemble (SVM + Naïve Bayes + Decision Tree):** Accuracy per fold ranges from 0.9121 to 0.9890. Average accuracy is 0.9429, lower than boosting methods but higher than a single Decision Tree. Mean F1 score is 0.9382, indicating balanced performance across classes.

Conclusions

- Boosting methods outperform individual models. AdaBoost, Gradient Boosting, and XGBoost all outperform a single Decision Tree. XGBoost achieved the highest average accuracy (0.9758).
- Random Forest improves over a single Decision Tree by aggregating multiple trees, reducing overfitting and variance. Its performance (0.9670) is competitive with boosting methods.

- Stacked ensemble provides moderate gains. Combining SVM, Naïve Bayes, and Decision Tree yields better performance than a single model but is still below top boosting algorithms. Including stronger base learners may improve performance.
- Hyperparameter tuning is critical. XGBoost and Gradient Boosting achieved their best performance using carefully tuned parameters. Small changes in parameters like learning rate, depth, and number of estimators significantly impact accuracy.
- Recommendation: For maximum predictive accuracy, XGBoost with chosen hyperparameters is recommended. For interpretable models with slightly lower accuracy, Random Forest or AdaBoost are good choices. Stacked ensembles can be explored further by including stronger base learners.